Deployment-Accuracy Gaps in Compressed TCNs for Real-Time Gait Phase Detection

Anonymous Author(s)
Affiliation withheld for double-anonymous review

***Abstract*—Wearable-based gait phase detection is a critical enabler for rehabilitation monitoring, fall-risk assessment, and assistive robotics, yet deploying deep learning models on resource-constrained microcontrollers remains challenging. This study evaluates INT8 post-training quantization and structured channel pruning for a Standard TCN classifying gait phases (LR, LS, PSw, Sw) from shank-mounted IMU data (NONAN GaitPrint), on the ESP32-C3, under subject-wise 5-fold cross-validation (K=5, three seeds; N=15 runs per config). TCN achieved the highest mean macro F1 among compared architectures ($90.87\% \pm 1.47\%$), significantly outperforming CNN1D ($+0.53$ pp, p=0.004) and statistically tying Transformer/LSTM+GRU (p=0.80/0.17). A PyTorch weight-only quantize-dequantize (QDQ) proxy suggests INT8 is lossless ($90.88\% \pm 1.47\%$ vs. FP32; p=0.30); the *actual deployed* full-integer TFLite model confirms this proxy is reasonable but not exact: plain INT8 costs a further 0.72 ± 0.60 pp ($p = 1.8 \times 10^{-4}$), growing monotonically with pruning severity to -1.87 pp, -3.56 pp, and -8.05 pp for INT8+Prune75/50/25—the proxy–deployment gap widens the more aggressively the network is compressed. Structured pruning, which involves no quantization and carries no such gap, reduces macro F1 by 1.47 pp at 50% retention (5-epoch fine-tune; $p<0.001$), recoverable to -0.53 pp with 50 epochs. PSw and LR (transition phases) remain the most compression-sensitive under both, though the real INT8 effect is modest (LR -1.6 pp, PSw -0.8 pp). Since PSw/LR each span only $\sim$50–100 ms of the gait cycle, we adopt a clinically motivated 100 ms real-time budget rather than the coarser 500 ms offline-evaluation window: only INT8+Prune50 (75.7 ms) meets it, while INT8 (279.8 ms), Prune50 (253 ms), Prune75 (510.8 ms), and FP32 (856 ms) do not—the smallest, most quantization-dependent config is also the only one fast enough for real time.**

***Index Terms*—TCN, model compression, post-training quantization, TFLite, deployment validation, structured pruning, gait phase detection, ESP32-C3, edge AI, wearable sensors**

I. Introduction

Accurate detection of gait phases—such as loading response, loading stance, pre-swing, and swing—is fundamental to applications in clinical rehabilitation, fall prevention, prosthetic control, and sports biomechanics. Recent advances in deep learning, particularly recurrent and convolutional architectures, have substantially improved gait phase classification accuracy using wearable inertial measurement unit (IMU) sensors. However, these models are typically designed and validated on desktop or server-class hardware, and their large parameter counts and floating-point computations make them impractical for direct deployment on low-power, low-memory microcontrollers used in real-world wearable devices.

Temporal Convolutional Networks (TCNs), which leverage dilated convolutions to capture long-range temporal dependencies with fewer parameters than recurrent architectures, offer a promising middle ground between accuracy and efficiency and can be further compressed via quantization and pruning to meet the strict memory and latency budgets of microcontroller-class hardware such as the ESP32-C3. While prior work has explored model compression broadly for time-series classification, little attention has been paid to how compression affects accuracy unevenly across gait phases. Transition phases such as Loading Response (LR) and Pre-Swing (PSw) are inherently more ambiguous due to overlapping biomechanical signal characteristics, and we hypothesize these are more vulnerable to information loss from aggressive quantization and pruning than steady-state phases such as Swing (Sw).

This study addresses this gap by:

1) Selecting and training a Standard TCN backbone for gait phase classification using shank IMU data from NONAN GaitPrint, with matched-capacity FP32 baselines for context;
2) Systematically applying INT8 quantization and structured pruning, quantifying their impact on phase-specific accuracy and F1-score rather than overall accuracy alone;
3) Deploying the compressed models on ESP32-C3 hardware and measuring on-device latency and SRAM headroom against a clinically motivated real-time budget;
4) Validating whether a common PyTorch-side weight-only quantization proxy predicts the *actual deployed* full-integer TFLite model's accuracy (N=15), showing this proxy–deployment gap widens as compression techniques are combined.

II. Literature Review

A. Gait Phase Detection Using Wearable IMU Sensors

Gait phase detection using body-worn IMU sensors has been extensively studied, with shank- and foot-mounted sensors commonly used for their high sensitivity to phase transitions; deep learning methods—1D CNNs, LSTM, hybrid LSTM+GRU, and, more recently, Transformer encoders—now dominate over classical threshold-based or hidden-Markov approaches. Using the same NONAN GaitPrint dataset and shank-mounted six-axis IMU configuration, Choi and Choi [1] directly compared a 1D CNN, a hybrid LSTM+GRU model, and a Transformer for four-class gait phase classification, reporting comparable performance (macro F1 $\approx$91–93%) with the Transformer attaining the highest score. The present study replicates those three architectures, plus a standard TCN, as

FP32 baselines under an identical protocol (Section III); our baselines score slightly lower (90.3–90.9%, Table I), plausibly because we cap all four at a shared hidden width of 32 for a capacity-matched comparison rather than tuning each individually as in [1]. This study targets a distinct research question left open by prior work: how INT8 quantization and structured pruning affect *phase-specific* accuracy, and how the resulting models perform when deployed on microcontroller-class hardware.

B. Temporal Convolutional Networks (TCNs) for Time-Series Classification

TCNs use causal, dilated convolutions to achieve large receptive fields with a fraction of the parameters required by recurrent architectures, while supporting parallelized computation [2], making them attractive for edge deployment. All architectures here share a common hidden width (32 channels) and 0.5 s causal windows, yielding 10–26K-parameter models sized for ESP32-C3 rather than server-scale capacity.

C. Model Compression Techniques: Quantization and Pruning

Post-training quantization reduces numerical precision (e.g., FP32→INT8), reducing model size and enabling faster, more energy-efficient inference on integer-optimized hardware [3]. Structured pruning removes entire channels or filters, yielding actual speedups on commodity hardware (unlike unstructured pruning, which needs sparse-matrix support) [4]. Both are widely studied for image and audio models, but their differential effect across distinct temporal phases of a periodic signal—such as gait phases—and the gap between simulated and deployed quantization accuracy, remain comparatively underexplored; this study addresses both directly.

D. Edge AI Deployment on Microcontrollers (ESP32-C3)

TensorFlow Lite Micro (TFLM) enables compact neural networks to run directly on microcontroller hardware with kilobyte-scale memory budgets [5]. The ESP32-C3 is a single-core RISC-V microcontroller (160 MHz, ≈400 KB SRAM [6]) that lacks the vector acceleration of higher-end Espressif parts, motivating its use here to stress-test compressed models.

III. Methodology

A. Dataset and Preprocessing

- **Dataset:** NONAN GaitPrint [7], 35 subjects total. Input channels are *bilateral* shank-mounted IMUs (accelerometer + gyroscope on each shank, 6 axes per leg, 12 total)—the system requires two sensors, one per shank.
- **Sampling rate:** Original 200 Hz signal downsampled to 100 Hz for all reported experiments, balancing temporal resolution against on-device resource constraints.
- **Labeling:** Gait cycle phases labeled as Loading Response (LR), Loading Stance (LS), Pre-Swing (PSw), and Swing (Sw), following the four-phase taxonomy of [1]; the window label is taken at its final (most recent) sample, matching causal, real-time inference where only past samples are available.
- **Data split:** Subject-wise stratified 5-fold CV (K=5, 24 train / 4 val / 7 test subjects per fold) with three seeds; validation subjects rotate per fold (re-sampled from the non-test pool) to avoid repeated early-stopping bias; no subject leakage across train/val/test.
- **Normalization:** Per-channel z-score normalization computed on the training set only.
- **Windowing:** Causal 0.5 s windows with train stride 5 and validation/test stride 1 (test windows overlap ≈98%). A per-subject *blocked* macro F1 (each test subject's own macro F1, averaged with equal weight per subject) gives $90.92\%\pm1.45\%$ (N=15) for the FP32 TCN, matching the pooled window-level $90.87\% \pm 1.47\%$ (Table I): overlap does not measurably inflate the headline metric.
- **Training:** Adam optimizer ($\mathrm{lr} = 10^{-3}$), batch size 512, up to 50 epochs; best checkpoint selected by validation macro F1.
- **Fair comparison design:** All architectures use hidden width 32, four-class output, identical windowing/optimization/CV splits, and capped capacity (∼10–26K params, ∼40–100 KB FP32) rather than maximizing accuracy with large networks.
- **Compression fine-tuning:** Pruning uses *training subjects only* (fixed epoch budget, no validation access) to avoid double-dipping on the FP32 validation set; runtime assertions verify zero subject overlap for every fold.
- **Evaluation:** Table I reports mean ± std macro F1 over K=5 folds×3 seeds (N=15 runs) with paired Wilcoxon tests; compression results (Table II), the fine-tune ablation (Table IV), and real deployed TFLite accuracy (Table III) use the same protocol. Fig. 1 illustrates one fold/seed (0/42); Table V reports ESP32-C3 latency for five deployable configs from that checkpoint.

Phase support is highly imbalanced: in the reference train/val/test split used to illustrate this (one representative split, not summed across the 5 CV folds), LS and Sw together account for $(1{,}274{,}499 + 1{,}016{,}857)/2{,}583{,}680 \approx 88.7\%$ of test windows, vs. only 11.3% for the transition phases LR and PSw combined; every fold's test split preserves approximately this same majority:minority ratio. To prevent the majority phases from dominating optimization, all models—including the four FP32 baselines and every compression fine-tuning stage—are trained with a class-balanced weighted cross-entropy loss:

$$\mathcal{L} = -\sum_{c=1}^{4} \alpha_c \, y_c \log \hat{y}_c, \qquad \alpha_c = \frac{N}{4 \cdot N_c}, \tag{1}$$

where N is the number of training windows, N_c is the number of windows labeled with phase c, y_c is the one-hot ground-truth indicator, and $\hat{y}_c$ is the softmax output probability for phase c. This directly motivates reporting *macro*, rather than micro-averaged, metrics throughout this paper.

B. TCN Architecture

We adopt a lightweight TCN variant of Bai et al. [2] as the deployment backbone: each residual block here uses a single

dilated causal convolution rather than the two per block in Bai et al., trading some representational capacity for a smaller memory/latency footprint on microcontroller-class hardware. The model consists of four such blocks (dilation rates 1, 2, 4, and 8) with residual connections, batch normalization, and a lightweight classification head (global average pooling followed by a dense softmax layer over the four phase classes). Each block computes a dilated causal convolution (kernel size k=3, dilation d, ReLU) so that each output depends only on present and past inputs. Stacking four such blocks yields a receptive field of 31 samples (0.31 s at 100 Hz), comfortably within the 0.5 s (50-sample) input window while keeping the parameter count near 11K. The model consumes causal 0.5-second windows (50 samples at 100 Hz). Channel width is deliberately kept narrow (32 hidden channels) to retain a microcontroller-deployable footprint ($\sim$45 KB FP32). Each causal convolutional block left-pads by $(k-1)d$ samples, applies the dilated Conv1D, batch norm, and ReLU, then adds a residual connection (with a 1×1 projection when channel counts differ); the stem ends in global average pooling and a dense softmax over the four phase classes.

C. Baseline Architectures for Context

For context, three FP32 baselines (CNN1D, LSTM+GRU, Transformer) were retrained under the same hidden width, windowing, and cross-validation protocol (Table I).

D. Model Compression Protocol

Post-training quantization (INT8). We apply symmetric, *per-tensor* 8-bit quantization separately to each convolutional and fully connected weight layer: a single scale is set from that layer's largest absolute weight, each weight rounded to $[-127, 127]$ and mapped back to floating point. This quantize–dequantize (QDQ) proxy is used for PyTorch-side test-set evaluation (Section IV). For ESP32 deployment, models are exported as full-integer TensorFlow Lite graphs via *ai_edge_quantizer* (100 calibration windows from training subjects), which by default quantizes weights the same way (symmetric, per-tensor TENSORWISE, 8-bit) and additionally quantizes *activations* asymmetrically, per-tensor; the real-vs-proxy gap (Section IV) is thus attributable to activation quantization, not a weight-granularity mismatch. Real deployed accuracy is measured via a TFLite interpreter with a saturating int8 cast ($[-128, 127]$ for activations, matching the firmware).

Structured channel pruning. The TCN stem uses 32 hidden channels, ranked by summed absolute weight magnitude across all stem convolutional blocks; the top-k are retained and removed elsewhere in the stem and classification head, then fine-tuned on *training subjects only* (Adam, lr 10^{-4}, class-balanced loss) before re-evaluation. Primary k-fold configs sweep 25/50/75% channel retention (Prune25/50/75; Prune25 *retains* 25% and is thus most aggressive, Prune75 mildest) with a default 5-epoch fine-tune, reported in Table II at N=15; Table IV separately ablates the fine-tune epoch budget at 50% retention.

Extended compression grid (k-fold). Beyond FP32/INT8 and Prune25/50/75, combined INT8+Prune50 populates the Pareto front (Fig. 1); the real-deployment dose-response comparison (Table III) additionally covers INT8+Prune25 and INT8+Prune75, giving a four-point severity grid. INT4 is excluded because full-integer INT4 TFLite is not supported on the ESP32-C3 deployment target. Fine-tune schedule ablation on Prune50 (5/15/50 epochs) isolates whether accuracy loss under pruning reflects insufficient recovery training rather than an inherent pruning limit (Table IV).

Compression variants comprise INT8 post-training quantization (PyTorch QDQ proxy for test metrics; full-integer TFLite PTQ for ESP32), structured 25/50/75% channel pruning with train-only fine-tuning, and combined INT8+Prune{25,50,75}.

E. Hardware Deployment (ESP32-C3)

After PyTorch compression evaluation, selected models are converted to TensorFlow Lite (.tflite) format and deployed via TensorFlow Lite Micro on an ESP32-C3 development board (160 MHz, Arduino IDE, Chirale_TensorFlowLite). On-device inference latency is measured per 0.5-second input window over 1000 timed trials (20 warmup invocations), alongside minimum free heap during benchmarking and a fixed 120 KB tensor arena.

F. Evaluation Metrics

Macro F1 is the unweighted mean of the four per-phase F1 scores. Unlike accuracy or a support-weighted average, this gives Loading Response (LR) and Pre-Swing (PSw) equal weight to the majority phases despite their $\sim 6\%$ and $\sim 5\%$ test-set support, respectively.

- Phase-specific F1 and accuracy for LR, LS, PSw, and Sw.
- Pareto analysis: macro F1 vs. model size (KB) per compression config.
- On-device latency (ms) and minimum free heap (KB) on ESP32-C3.
- **Real-time budget:** a clinically motivated **100 ms** inference budget, since Loading Response and Pre-Swing each span only $\sim$50–100 ms of a $\sim$1 s gait cycle [8]—tighter than the 500 ms a naive non-overlapping 0.5 s window would otherwise allow (Section IV-E).

IV. Results

A. Baseline Comparison and Model Complexity

Table I compares TCN against three FP32 baselines under the primary K=5 $\times$ 3 protocol (mean $\pm$ std over 15 runs). Macro F1 scores cluster tightly (90.34–90.87%); TCN attains the highest mean macro F1 ($90.87\% \pm 1.47\%$). Paired Wilcoxon tests on matched fold$\times$seed pairs show TCN statistically equivalent to Transformer (Δ=+0.04 pp, p=0.80) and LSTM+GRU (+0.28 pp, p=0.17), but significantly higher than CNN1D (+0.53 pp, p=0.004).

TABLE I
BASELINE FP32 COMPARISON (K=5 × 3 SEEDS)

Model	Params	KB	Macro F1 (%)	Phase F1 (%)			
				LR	LS	PSw	Sw
TCN	11,560	45.2	**90.87 ± 1.47**	**85.81**	**97.78**	82.72	97.18
Transformer	25,956	101.4	90.83 ± 0.99	85.37	97.73	**82.95**	**97.27**
LSTM+GRU	12,356	48.3	90.59 ± 1.19	84.95	97.70	82.52	97.22
CNN1D	10,727	41.9	90.34 ± 1.21	84.32	97.52	82.54	97.00

Mean ± std macro F1 (N=15); phase F1 reported as means. Hidden width 32 for all models.

TABLE II
TCN K-FOLD COMPRESSION (MACRO F1 %, MEAN ± STD; N=15 PER CONFIG)

Config	Macro F1	PSw F1	Wilcoxon p vs. FP32
FP32	90.87 ± 1.47	82.72 ± 2.54	—
INT8	90.88 ± 1.47	82.75 ± 2.54	0.303
Prune25	87.62 ± 1.45	78.97 ± 1.28	6.10e-05
Prune50	89.40 ± 1.43	81.13 ± 1.97	6.10e-05
Prune75	90.17 ± 1.30	81.90 ± 2.18	6.10e-05
INT8+Prune50	89.33 ± 1.36	80.93 ± 2.00	1.22e-04

Paired Wilcoxon signed-rank tests on matched fold×seed pairs (N=15) vs. FP32. All configs complete (120/120 jobs). p=6.10e−05 is the N=15 Wilcoxon floor (see Limitations). Config names denote % channels retained; Prune25 is most aggressive, Prune75 mildest.

B. TCN Compression Under K-Fold Cross-Validation

Table II reports the primary compression results aggregated over N=15 matched fold×seed pairs, with paired Wilcoxon signed-rank tests against the FP32 reference (same checkpoints, no retraining).

The PyTorch QDQ proxy is a reasonable, but imperfect, stand-in for deployed INT8 accuracy. INT8 attains $90.88\% \pm 1.47\%$ macro F1 (+0.01 pp vs. FP32; Wilcoxon p=0.30) with an estimated payload of ∼13.0 KB (∼3.4× smaller than FP32) under the PyTorch weight-only quantize-dequantize (QDQ) proxy used throughout Table II. This proxy quantizes only weights; it does not model activation quantization noise. We therefore evaluated the *actual deployed* full-integer TFLite artifacts (Section IV-E) against the complete stride-1 test set (not a subsample) using the TFLite interpreter with a saturating int8 cast matching the ESP32-C3 firmware, across the same primary protocol as the rest of this paper (N=15: 5 folds × 3 seeds; Table III).

As a control, we ran the identical export-and-evaluate pipeline with *no* quantization (FP32 TFLite, Table III): real and PyTorch accuracy agree to under 0.0001 pp across all 15 fold×seed pairs, i.e., float roundtrip noise only. This confirms the pipeline itself introduces no artifact, so the gaps below reflect quantization, not an export or evaluation bug. The real deployed INT8 model loses a further $0.72\% \pm 0.60\%$ macro F1 beyond the proxy's estimate ($p=1.8 \times 10^{-4}$ vs. FP32)—small in absolute terms but consistent enough across all 15 fold×seed pairs to be statistically significant. Combining INT8 with pruning reveals a monotonic dose-response trend, not just a single larger gap: the real-vs-FP32 cost grows from −0.72 pp (INT8 alone) to −1.87 pp (INT8+Prune75) to −3.56 pp (INT8+Prune50) to −8.05 pp (INT8+Prune25), and the real-vs-proxy cost follows the same order (−0.73→ −1.18→ −2.02→ −4.82 pp)— the more aggressively the network is pruned, the harder activation quantization noise is for the weight-only proxy to anticipate. INT8+Prune25's mean conceals wide spread (±4.92 pp) driven by one extreme worst case (Table III). Per-phase (mean over N=15), real INT8 degrades LR somewhat more ($85.81\% \rightarrow 84.18\%$, −1.6 pp) than Sw ($97.18\% \rightarrow 96.99\%$, −0.2 pp), and this pattern is more pronounced for INT8+Prune50 (LR −7.6 pp vs. Sw −0.9 pp)—consistent with the transition-phase vulnerability identified under pruning, though at a modest scale for plain INT8. **Accuracy claims for quantized models should still be validated on the exact deployed artifact**, but the QDQ proxy used here is not unreasonable for single-technique INT8 quantization.

TABLE III
REAL DEPLOYED TFLITE VS. PYTORCH ACCURACY (MACRO F1 %, TCN, N=15)

Config	Real TFLite	Δ vs. FP32	Δ vs. proxy	Wilcoxon p
FP32 (control)	90.87 ± 1.52	−0.00 ± 0.00	n/a	0.225
INT8	90.15 ± 1.35	−0.72 ± 0.60	−0.73 ± 0.61	1.83e-04
INT8+Prune75	89.00 ± 1.50	−1.87 ± 1.24	−1.18 ± 1.06	1.22e-04
INT8+Prune50	87.32 ± 2.32	−3.56 ± 2.10	−2.02 ± 1.80	6.10e-05
INT8+Prune25	82.82 ± 4.92	−8.05 ± 5.03	−4.82 ± 4.89	6.10e-05

Real TFLite: full test set (stride-1), exact exported artifact, 5 folds × 3 seeds. FP32/INT8/INT8+Prune50 proxy from Table II (INT8+Prune50's proxy independently re-derives pruning/fine-tuning, Section V-E); INT8+Prune25/75 proxy applies QDQ to the existing checkpoint directly (no re-finetune). FP32 (control): same pipeline, no quantization—real vs. PyTorch agree to < 0.0001 pp. Δ is the mean of paired per-fold×seed differences; rows ordered by pruning severity (Prune75 mildest, Prune25 most aggressive)—both Δ columns grow monotonically. Worst case: INT8+Prune25 fold 2/seed 456 (67.43%, −24.04 pp, PSw F1 = 0.29 from LS→PSw confusion at hidden width 8), driving this config's ±4.92 pp std; other pairs stay 78–88%.

Pruning severity trades size for accuracy in a monotonic pattern. Unlike INT8, pruning involves no quantization and carries no proxy–deployment gap. Prune25 (∼4.3 KB estimated payload) reduces macro F1 by 3.25 pp ($87.62\% \pm 1.45\%$, p<0.001), with the largest phase-specific loss in LR (−6.98 pp) and PSw (−3.75 pp). Prune50 (∼13.1 KB estimated payload) costs 1.47 pp ($89.40\% \pm 1.43\%$, p<0.001). Prune75 (∼26.4 KB estimated payload) costs 0.70 pp ($90.17\% \pm 1.30\%$, p<0.001). All pruned configs remain significantly below FP32 at α=0.05, but the effect size shrinks as more channels are retained.

Fig. 1 shows the *estimated-payload* Pareto front built from QDQ-proxy accuracy (Table V has measured sizes). Prune75 has the best no-quantization accuracy (∼26.4 KB, −0.70 pp) and INT8 nearly matches it with a small, well-characterized real gap ($90.15\% \pm 1.35\%$), but under the clinically motivated 100 ms budget (Section III), neither is fast enough (510.8 ms, 279.8 ms). Only INT8+Prune50 (75.7 ms) resolves transitions at this rate, at a larger real accuracy cost ($87.32\% \pm 2.32\%$)—accuracy alone does not predict deployability, and the fastest config is also the least accurate one.

C. Fine-Tuning Schedule Ablation

Table IV isolates whether accuracy loss under Prune50 reflects an insufficient post-prune recovery budget rather than an inherent pruning limit. With the default 5-epoch fine-tune, Prune50 macro F1 is 1.47 pp below FP32 (p<0.001);

TABLE IV
PRUNE50 FINE-TUNING SCHEDULE ABLATION (K-FOLD; N=15; WILCOXON VS. FP32)

Config	FT ep.	Macro F1	Δ vs. FP32	PSw F1	p
Prune50	5	89.40 ± 1.43	-1.47	81.13 ± 1.97	6.10e-05
Prune50 (15 ep)	15	89.83 ± 1.54	-1.04	81.29 ± 2.79	6.10e-04
Prune50 (50 ep)	50	90.34 ± 1.11	-0.53	82.23 ± 1.82	0.015

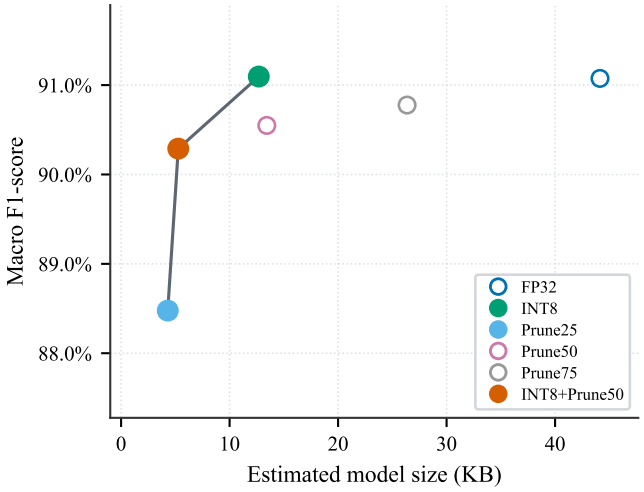

Fig. 1. Illustrative Pareto front (fold 0, seed 42). Horizontal axis is *estimated* payload, not measured TFLite size (Table V); k-fold aggregates in Table II. Frontier: Prune25 → INT8+Prune50 → INT8; Prune50/Prune75/FP32 are dominated here. Prune25 *retains* only 25% of channels (most aggressive); Prune75 is mildest. The TFLite flatbuffer/runtime format adds a roughly constant ∼8 KB overhead over the estimated payload for every config (Table V minus this figure's x-axis), so estimated and measured sizes convert between each other by this offset.

extending to 15 epochs recovers 0.43 pp (p=6.1×10^{-4}), and 50 epochs recovers 0.94 pp (p=0.015). PSw F1 improves from $81.13\% \pm 1.97\%$ (5 ep) to $82.23\% \pm 1.82\%$ (50 ep), approaching FP32 ($82.72\% \pm 2.54\%$). The residual gap at 50 epochs (−0.53 pp, still significant) indicates that structured 50% channel removal retains information loss beyond what fine-tuning can recover.

D. Compression Trade-offs and Phase Degradation

Fig. 1 qualitatively illustrates the accuracy–size trade-off on a single fold×seed pair (fold 0, seed 42) under the QDQ proxy, consistent with the k-fold aggregates in Tables II and IV *as proxy accuracy*. Table III shows the real deployed INT8 artifact tracks this proxy closely, with a small additional cost that grows once pruning is combined with quantization—this Pareto view is a reasonable but slightly optimistic guide for plain INT8, and more so for INT8+Prune50. On this same fold×seed pair, phase *accuracy* can remain nearly flat while phase F1 falls under compression (INT8+Prune50: LR accuracy +0.01 pp vs. LR F1 −1.318 pp; PSw F1 −1.324 pp), and structured pruning reduces PSw/Sw accuracy by up to −1.8 pp—motivating macro and phase F1 over accuracy alone throughout this paper.

TABLE V
ON-DEVICE INFERENCE ON ESP32-C3 (STANDARD TCN, FOLD 0, SEED 42)

Config	Mean (ms)	p95 (ms)	TFLite (KB)	Heap (KB)
FP32	856.2	856.2	52.1	116.3
Prune75	510.8	510.8	34.5	156.3
INT8	279.8	279.8	21.3	156.3
Prune50	253.0	253.0	21.6	156.3
INT8+Prune50	75.7	75.7	13.4	156.3

ESP32-C3 @ 160 MHz; TFLite Micro. Latency/0.5 s window (N=1000, 20 warmup). Mean and p95 round to the same value because latency is essentially deterministic on this bare-metal, single-core target: max−min jitter across all 1000 trials is < 0.08 ms for every config. Heap = min. free dynamic heap; arena 120 KB (all but FP32) or 160 KB (FP32); weights in flash. Only INT8+Prune50 meets the clinically motivated 100 ms real-time budget (Section IV-E).

E. On-Device Performance on ESP32-C3

Table V reports on-device latency for five TFLite exports derived from the fold 0, seed 42 compression checkpoints. Benchmarks replay one fixed normalized $(50, 12)$ window in a tight loop (N=1000 trials after 20 warmup invocations) without live IMU or BLE I/O.

A non-overlapping 0.5 s window (hop = window) allows up to 500 ms per inference, but this cadence cannot resolve a transition within the window it is detected in; against the clinically motivated **100 ms** budget (Section III), only INT8+Prune50 (75.7 ms) qualifies—and even then with only 24.3 ms of headroom left for sensor read, preprocessing, and any BLE transmission, a tight margin rather than a comfortable one. Prune50 (253 ms), INT8 (279.8 ms), Prune75 (510.8 ms), and FP32 (856 ms) all miss the 100 ms budget—the fastest config is also the least accurate one (Section V). The stride-1 (10 ms hop) offline protocol (Section III) would need sub-10 ms inference, which no config achieves either. FP32 additionally requires a 160 KB tensor arena, reducing minimum free heap to 116 KB vs. 156 KB for the 120 KB-arena configs.

V. DISCUSSION

A. Variance Across Folds and Seeds

Across the K=5 folds×3 seeds (N=15) baseline runs, σ_{fold}=1.38 pp exceeds σ_{seed}=0.28 pp for TCN, so performance spread (Table I) primarily reflects which subjects appear in each test fold rather than initialization noise. This persists after real deployment: the single worst fold×seed pair in Table III is fold 4 for *both* INT8 (seed 456) and INT8+Prune50 (seed 123)—milder than, but consistent with, the fold-level variance already visible at FP32.

B. Vulnerability of Transition Phases

Across k-fold compression (Table II), LR F1 shows the largest pruning-related degradation relative to FP32 (−6.98 pp Prune25, −3.08 pp Prune50 5 ep, −3.11 pp INT8+Prune50); PSw is also clearly sensitive but consistently about half of LR's loss (−3.75 pp, −1.59 pp, −1.79 pp). INT8 alone changes PSw by only +0.03 pp (p=0.30), and the fine-tune ablation confirms only partial PSw/LR recovery. This may be confounded with support/difficulty, since LR/PSw are the

only transition phases and lowest-support classes ($\sim$6%/5%); still, relative to each phase's own FP32 F1 under Prune25, LR/PSw lose more (-8.1%, -4.5%) than steady-state LS/Sw (-1.3%, -1.1%)—not purely a baseline-F1 artifact. The same ordering reappears, more mildly, under real deployed compression (Table III): LR falls -1.6 pp (INT8) and -7.6 pp (INT8+Prune50), PSw falls -0.8 pp and -4.5 pp, vs. only -0.2/-0.9 pp for steady-state Sw.

C. Weight- vs. Activation-Quantization Tolerance

INT8 weight quantization alone preserved macro F1 within 0.01 pp of FP32 (p=0.30), so TCN's dilated receptive field and narrow hidden width tolerate 8-bit *weight* precision well. The real deployed model additionally quantizes *activations*, costing a further 0.72 ± 0.60 pp—small but statistically significant across all 15 pairs (Table III), indicating some activation noise compounds across the four dilated blocks that weight quantization alone does not show. This grows monotonically with pruning severity—-1.87 pp (Prune75), -3.56 pp (Prune50), -8.05 pp (Prune25), i.e., roughly $2.6\times$ to $11\times$ plain INT8's cost (Table III); one plausible mechanism is that pruned channels leave less redundancy to absorb activation noise, though we did not test this directly. Pruning alone removes capacity by a different, gap-free mechanism (Prune50 0.53 pp below FP32 even at 50-epoch fine-tuning), but PSw suffers under both.

D. Efficiency-Accuracy Trade-off

Higher accuracy does not imply faster inference. At a lenient 500 ms (non-overlapping window) budget, INT8 (real $90.15\%\pm1.35\%$, 279.8 ms) nearly matches Prune75's accuracy while staying real-time; but PSw/LR's $\sim$50–100 ms duration motivates the stricter 100 ms budget (Section III), which only INT8+Prune50 meets, at a real accuracy cost roughly $5\times$ plain INT8's—and carries the highest fold$\times$seed standard deviation of any non-Prune25 config in Table III (±2.32 pp vs. ±1.35 pp for INT8): the one config fast enough for real time is also among the least predictable.

E. Limitations

- Results derive from a single public dataset of healthy young adults at a single self-selected walking speed (NONAN GaitPrint [7]); generalization to other walking speeds, pathological gait, or age groups is untested. ESP32-C3 benchmarks used a synthetic replay window (fold 0, seed 42) with no live IMU/BLE.
- Test stride 1 yields highly overlapping windows; per-subject blocked macro F1 (matching the pooled window-level number to within noise, Section III) mitigates this, but Wilcoxon tests use fold$\times$seed aggregates (N=15, not fully independent), bounding the smallest signed-rank p at 6.10×10^{-5}; Holm–Bonferroni correction changes no conclusion.
- INT8+Prune50's PyTorch proxy independently re-derives pruning/fine-tuning rather than reusing the standalone Prune50 checkpoint ($\sim$0.3 pp of fine-tune-trajectory variance beyond the quantization effect); INT8+Prune25/75's proxies reuse the existing checkpoint directly, avoiding this.
- INT8 PTQ calibration used 100 windows; a sweep (fold 0/seed 42) found plain INT8 stable (90.7–90.9% at 100/500/1000 windows) but INT8+Prune50 non-monotonic (85.9–88.4%)—not fully stress-tested for combined configs. Power (mW) was not measured; latency used TFLite Micro reference kernels (Chirale_TensorFlowLite).

VI. Conclusion

This study evaluated how INT8 quantization and structured pruning affect phase-specific accuracy in a Standard TCN for gait phase detection. TCN achieved $90.87\% \pm 1.47\%$ macro F1 under K=5 $\times$ 3 cross-validation, the highest mean among compared architectures while remaining microcontroller-deployable. A PyTorch weight-only QDQ proxy showed INT8 statistically equivalent to FP32, but the *actual deployed* full-integer TFLite model revealed a real cost that grows monotonically with pruning severity, from -0.72 pp (INT8 alone) to -8.05 pp (INT8+Prune25): the proxy–deployment gap widens the more aggressively the network is compressed. PSw/LR remained the most compression-sensitive phases throughout, and since they span only $\sim$50–100 ms of the gait cycle, we adopt a clinically motivated 100 ms real-time budget: only INT8+Prune50 (75.7 ms) meets it, exposing a genuine tension between the one config fast enough for real time and the higher, more reliable accuracy of the slower alternatives. Future work will validate live IMU streaming and additional cohorts.

References

[1] W. Choi and M.-T. Choi, "Deep learning-based gait phase detection using shank-mounted IMU data: Classification approach," *PLOS ONE*, vol. 21, no. 4, p. e0344002, 2026, doi: 10.1371/journal.pone.0344002.

[2] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[3] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[4] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[5] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.

[6] Espressif Systems, *ESP32-C3 Technical Reference Manual*, version 1.3, 2024.

[7] T. M. Wiles et al., "NONAN GaitPrint: An IMU gait database of healthy young adults," *Scientific Data*, vol. 10, no. 867, 2023, doi: 10.1038/s41597-023-02704-z.

[8] J. Perry and J. M. Burnfield, *Gait Analysis: Normal and Pathological Function*, 2nd ed. Thorofare, NJ, USA: SLACK Incorporated, 2010.